

Budgeted End-to-End LLM4HLS Agent: Automated Repair and Optimization of Vitis HLS Code within Constrained Tool-Call Budgets

Anonymous

Submitted to FPT’26 Design Competition – Track A: LLM4HLS Agent

Abstract—We present an autonomous AI agent that repairs and optimizes AMD Vitis HLS C/C++ kernels under a constrained tool-invocation budget. The agent receives an HLS task—possibly containing a functional bug, a synthesis failure, a co-simulation deadlock, or merely an unoptimized baseline—and iteratively drives it to functional correctness before optimizing its PPA (Performance–Power–Area). The architecture follows a linear-with-backtracking three-stage flow: *correctness repair* → *synthesis verification* → *PPA optimization*, where every code edit re-verifies already-passed stages and a monotone checkpoint archive provides rollback for free. The optimization stage implements the AMD LLM4HLS four-phase workflow: design-brief extraction, strategy proposal with compatibility annotations, a reviewer-AI dual-confirmation gate, and combined code generation. A deterministic mechanical-check gate complements LLM self-review to catch signature changes that the LLM misses. We evaluate on six tasks spanning three task types (repair, structural, optimize) on Vitis 2025.2 targeting Alveo U55C at 200 MHz with all three competition-recommended open-weight models (DeepSeek V4 Pro, Qwen3.5 122B, Qwen3.6 27B): every model clears the csim correctness gate on all six tasks (18/18), demonstrating that the architecture transfers across models without modification. With DeepSeek V4 Pro the agent achieves two perfect scores and up to 205× latency reduction; all candidates close timing with conservative Fmax of 200–353 MHz at a maximum resource utilization of 3.5%. A graded token-mode A/B study shows 8–31% token savings without score degradation. All artifacts are packaged as a reproducible Docker image, and all experiment data are archived for auditability.

Index Terms—High-Level Synthesis, LLM agents, Vitis HLS, FPGA, correctness-first optimization, budgeted tool use

I. INTRODUCTION

Writing correct HLS code requires simultaneously managing algorithm correctness and hardware constraints; a misplaced `#pragma` can cause synthesis failure or co-simulation deadlock. Directly prompting an LLM to “write correct HLS” in one shot rarely succeeds; an iterative loop with tool feedback is necessary [1], [2], [3]. FPT’26 Track A formalizes this: build an autonomous agent that, under a bounded tool-call budget, repairs and optimizes HLS tasks [4]. Scoring uses a hidden testbench with a correctness gate (Eq. 1): correctness carries 0.5 weight (hard gate), synthesis 0.2, and PPA acceleration (capped at 8×, latency-only) 0.3. Token consumption is an explicit evaluation factor.

Our contributions: (1) a **three-stage architecture** (correctness → synth → optimize) with backtracking re-verification and a monotone checkpoint archive (§II); (2) an **AMD four-phase optimization workflow**—design-brief extraction, strategy proposal, reviewer-AI dual-confirmation, combined application [5] (§C); (3) a **dual-gate review** where deterministic mechanical checks catch signature changes that LLM self-review misses; (4) a **graded token-saving switch** saving 8–31% tokens without score loss; (5) **multi-model validation** on six tasks with three competition-recommended open-weight models.

II. SYSTEM ARCHITECTURE

The agent follows a *linear flow with backtracking re-verification and checkpoint arbitration* (Fig. 1). A router reads `task.toml` at zero credit cost and selects the stage path. The checkpoint maps stage completion to a monotone level (Lv0: nothing → Lv1: csim+pass → Lv2: +synth pass), with three archive rules: level-up → accept; same-level → lower latency wins; level-down → reject (auto-rollback). Every code edit passes a dual gate: deterministic mechanical checks (signature/include) followed by LLM self-review. Stage 1 runs a repair loop (csim fail → feedback → KB retrieval → LLM repair → dual-gate → retry). Stage 2 re-verifies csim before each synth (1 credit < 4 credits). Stage 3 implements the AMD four-phase workflow: extract design brief → propose strategies with compatibility annotations → reviewer-AI dual-confirmation → combined application, with credit-driven stopping. A post-optimization cosim re-check with repair-then-rollback guards against shipping deadlocks. Detailed algorithms, stopping policies, and the knowledge base (22 bug→fix entries) are in Appendix A–O.

III. EXPERIMENTS

All runs use Vitis 2025.2 [6] targeting Alveo U55C `xcu55c-fsvh2892-2L-e` @ 200 MHz (5 ns). Scoring uses the official `grade()` on hidden testbenches. We evaluate three competition-recommended models [4], [7], [8]: DeepSeek V4 Pro (1.6T MoE), Qwen3.5 122B A10B, and Qwen3.6 27B, on six tasks (three official + three self-built). Setup details and the task suite are in Appendix D.

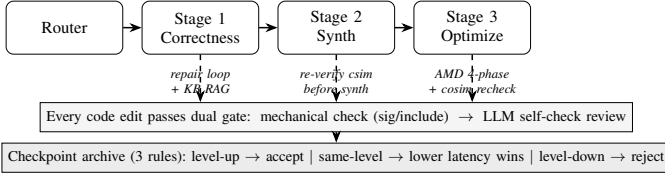


Fig. 1: Agent architecture. Three stages run sequentially; every code edit passes a dual review gate before the checkpoint arbitrates. The monotone-level rule gives automatic rollback.

A. Main Results (DeepSeek V4 Pro)

TABLE I: End-to-end results (DeepSeek V4 Pro, best archived run). Correctness 6/6 = 100%.

Task	Diff.	SCORE	Cand lat.	Accel.	Cr.	Tokens
projection	2	1.400	0 [†]	—	5	3,629
residual	4	4.000	10	13.5×	65	77,754
dotProduct	3	3.000	20	51.35×	40	67,253
matmul	3	2.691	3,126	5.25×	35	50,729
fir	2	2.000	542	30.5×	40	127,162
vecadd	1	1.000	20	205×	20	46,924

[†] Combinational logic (latency=0); PPA slice unreachable (§M).

Two tasks achieve perfect scores (residual 4.000, dotProduct 3.000). All candidates close timing (Fmax 200–353 MHz, well above the 100 MHz rule) at $\leq 3.5\%$ resource utilization. Full PPA data is in Appendix G.

B. Multi-Model Comparison (Contest Rule 6)

TABLE II: Multi-model comparison (one run per model/task, token-mode full).

Task	DeepSeek V4 Pro			Qwen3.5 122B			Qwen3.6 27B		
	Scr	Cr	Tok	Scr	Cr	Tok	Scr	Cr	Tok
projection	1.4	5	3.6k	1.4	5	15.1k	1.4	5	13.9k
residual	4.0	65	77.8k	3.1	30	42.0k	3.4	60	68.4k
dotProduct	3.0	40	67.3k	3.0	15	72.8k	2.3	10	38.5k
matmul	2.7	35	50.7k	2.2	10	67.5k	3.0	30	54.5k
fir	2.0	40	127.2k	2.0	30	152.2k	1.5	25	64.3k
vecadd	1.0	20	46.9k	0.5	20	59.7k	0.7	20	37.7k
Total	14.1 / 373k			12.2 / 409k			12.3 / 277k		

All three models clear the csim correctness gate on all six tasks (18/18)—the correctness-first architecture is model-agnostic. Optimization quality varies: Qwen3.6 27B produced the only perfect matmul (210×) while stalling on vecadd; DeepSeek achieves the best score/token trade-off. A graded token-mode switch (balanced) saves 8–31% tokens without score loss (Appendix G). Variance and failure-mode analysis is in Appendix G.

IV. DISCUSSION

The scored PPA term is $p = \min(\text{Accel}, 8)/8$ —latency-only. Under this metric, latency-first optimization is score-optimal; area/power reduction has zero marginal value. We guard against the two failure modes: area explosion is caught

by the reviewer-AI feasibility gate, and timing failure is prevented by the 5 ns synthesis target. Detailed design rationale, a harness edge case (zero-latency scoring), limitations, and future work are in Appendix J.

V. RELATED WORK

ReAct [1] interleaves reasoning with tool actions; our agent instantiates this loop in the HLS domain. HLS-Repair [3] and HlSPilot [9] repair/automate HLS with LLMs but stop at correctness; we couple repair with PPA optimization under a hard budget. AMD’s case study [5] proposes a human-in-the-loop four-phase workflow; we engineer it into a fully autonomous loop. Extended related work is in Appendix J.

VI. CONCLUSION

We built a budgeted end-to-end LLM4HLS agent achieving 100% correctness (18/18 across three models), two perfect scores, and up to 205× latency reduction on six tasks with Vitis 2025.2 + Alveo U55C. The core design—three-stage architecture, AMD four-phase optimization, dual-gate review, and graded token saving—transfers across models without modification and is packaged as a reproducible Docker sub-mission.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [2] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, “Autochip: Automating hdl generation using llm feedback,” *arXiv preprint arXiv:2311.04887*, 2023.
- [3] K. Xu, G. L. Zhang, X. Yin *et al.*, “Automated c/c++ program repair for high-level synthesis via large language models,” *arXiv preprint arXiv:2407.03889*, 2024.
- [4] FPT’26 Design Competition Organizing Committee, “Fpt’26 design competition, track a: Llm4hls agent — submission guidelines,” 2026, official task description and submission rules.
- [5] AMD, “Llm-aided hls dataflow optimization: A case study,” <https://www.amd.com/en/developer-resources/llm-aided-hls-dataflow-optimization>, aMD developer case study, accessed 2026-07.
- [6] —, “Vitis high-level synthesis user guide (ug1399),” <https://docs.amd.com/r/en-US/ug1399-vitis-hls>, version 2025.2, accessed 2026-07.
- [7] DeepSeek-AI, “Deepseek-v3 technical report,” 2024.
- [8] Qwen Team, “Qwen3 technical report,” 2025.
- [9] C. Xiong, C. Liu, H. Li *et al.*, “Hlspilot: Llm-based high-level synthesis,” *arXiv preprint arXiv:2408.06810*, 2024.
- [10] J. Yang, C. E. Jimenez, A. Wettig *et al.*, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *arXiv preprint arXiv:2405.15793*, 2024.
- [11] Anthropic, “Building effective agents,” <https://www.anthropic.com/research/building-effective-agents>, accessed 2026-07.
- [12] Y.-D. Tsai, M. Liu, and H. Ren, “Rtlfixer: Automatically fixing rtl syntax errors with large language models,” *arXiv preprint arXiv:2311.16543*, 2023.

APPENDIX

A. Stage 1: Correctness Repair Loop

Goal: pass all correctness-gate stages. Each iteration:

- 1) **Pre-csim LLM review** (first attempt only): before spending a credit on csim, let the LLM inspect the code and fix obvious bugs. On the `residual_stream_deadlock`

task, this step identified and fixed the DATAFLOW deadlock pattern *before* any tool call—saving a 20-credit failed cosim and a 15-minute timeout.

- 2) Run csim (+ cosim if structural).
- 3) Pass → archive to Lv1, proceed to Stage 2.
- 4) Fail → distill feedback → KB retrieval → repair → dual-gate review → retry.

Dual-gate review (applied at every code edit):

- *Gate 1 – Mechanical checks* (deterministic, non-LLM): verify the top-level function signature is unchanged and the header `#include` is still present. In one observed instance, the LLM self-check approved a candidate whose top-level signature had changed (which would fail hidden-testbench linking); the mechanical gate caught it.
- *Gate 2 – LLM self-check review*: same model, reviewer prompt, checking interface/new bugs/pragma hazards. A rejection triggers re-generation.

B. Stage 2: Synthesis Verification

Goal: verify synthesizability and obtain baseline latency. On synth failure, a repair loop runs, but each edit re-verifies csim (1 credit) before spending another synth (4 credits). The loop runs up to `max_synth_rounds=3`. A latency validity defense treats ≤ 0 as missing data, preventing a “0-cycle” design from corrupting the archive.

C. Stage 3: PPA Optimization — AMD Four-Phase Workflow

a) Phase 1 – Context Loading.: Official design document + design brief extraction (LLM#3, cached) + synthesis report + device constraints + checkpoint snapshot.

b) Phase 2a – Strategy Proposal (LLM#4).: Proposes 2–4 strategies, each annotated with `combinable_with`. Uses JSON structured output.

c) Phase 2b – Reviewer AI (LLM#5).: Two steps: (1) feasibility review per strategy (interface unchanged, functionality preserved, synthesizable, resources fit); (2) compatibility re-check. The *proposer* and *reviewer* must both agree before combination.

d) Phase 3 – Combined Application (LLM#6).: Selected subset applied as one candidate, passing dual-gate review, then re-verifying csim + synth.

Stopping policy (credit-driven): convergence (not strictly faster → stop); credit exhaustion; credit reservation for structural tasks (26 credits for final cosim re-check); safety round cap (20).

D. Post-Optimization Co-Simulation Re-check

After optimization, a final RTL sanity check runs for all task types. If best changed and cosim is affordable: pass → mark `cosim_ok`; fail → repair-then-rollback (repair with cosim feedback, up to 3 retries, then rollback to pre-optimization snapshot).

a) Hardware and software.: Ubuntu 22.04 server (8-core / 32 GB), Vitis 2025.2, csim/synth/cosim via `vitis-run -mode hls`. Target: Alveo U55C `xcu55c-fsvh2892-2L-e`, 200 MHz (5 ns) clock. Scoring uses hidden testbenches outside the agent budget. All raw artifacts archived in `experiments/`.

b) Models.: DeepSeek V4 Pro (1.6T MoE, FP8; official API), Qwen3.5 122B A10B (AWQ-4bit), Qwen3.6 27B (FP8) (both via OpenAI-compatible MaaS). Temperature 0.2, token-mode `full` unless stated.

c) Protocol.: Each (model, task) pair run once; DeepSeek V4 Pro additionally has repeat runs for variance analysis. One exception: first Qwen3.5 matmul run died to LLM-API timeout and was re-run once with longer timeout.

TABLE III: Task suite. First three are official public tasks; last three are self-built.

Task	Type	Diff.	Bud.	Initial condition
projection	repair	2	20	csim fails: angle==0 branch drops vertex term
residual	struct.	4	80	csim ok, cosim deadlocks (FIFO burst)
dotProduct	optimize	3	40	correct, no pragmas
matmul	optimize	3	60	32×32 matmul, no pragmas
fir	optimize	2	40	32-tap FIR, no pragmas
vecadd	optimize	1	20	vector add, no pragmas

E. Metered Tool Interface

The harness provides ToolServer with three budget-charged calls: `csim=1 credit`, `synth=4 credits`, `cosim=20 credits`. Each task’s budget is in `task.toml`. Credits do not accumulate across tasks.

F. Scoring Formula

$$\text{score} = \begin{cases} 0 & \text{if not functional_pass} \\ \text{diff} \times (0.5c + 0.2s + 0.3p) & \text{otherwise} \end{cases} \quad (1)$$

where $c = 1$ if hidden csim (+cosim if required) passes, $s = 1$ if synthesizable, $p = \min(\text{Accel}, 8)/8$.

G. Task Types

TABLE IV: Task types and correctness gates.

task_type	Typical initial state	Correctness gate
repair	csim fails (functional bug)	csim
optimize	csim passes, just slow	csim
structural	csim passes, cosim deadlocks	csim + cosim
generate	generate from scratch	csim (+cosim)

~89% of completion tokens are reasoning tokens in reasoning models—the dominant cost. We implemented a graded `--token-mode` switch:

TABLE V: Full-PPA report of submitted candidates (DeepSeek V4 Pro), from Vitis csynth reports. Conservative Fmax = 1000/(est + unc); all meet timing (≥ 100 MHz).

Task	Lat. (cyc)	Est. clk (ns)	Fmax (MHz)	LUT	FF	BRAM-18K	DSP	Timing
projection	0	2.54	257	692 / 1.30M (0.05%)	0	0 / 4032	0 / 9024	✓
residual	10	1.48	353	366 (0.03%)	14	0	0	✓
dotProduct	20 [‡]	3.17	221	1,809 (0.14%)	1,135	0	64 (0.7%)	✓
matmul	3,126	3.65	200	14,847 (1.1%)	18,018	10 (0.2%)	158 (1.8%)	✓
fir	542	2.98	231	19,033 (1.5%)	27,041	0	316 (3.5%)	✓
vecadd	262 [§]	2.98	231	3,612 (0.3%)	3,377	0	32 (0.4%)	✓

[‡]Latency 20 is best archived run; PPA row from latest run (identical structure). [§]vecadd from clean re-run.

TABLE VI: Token-mode policy.

Mode	Settings
full (default)	No effort override
balanced	review=off, select=off
aggressive	Above + brief=off, propose=off

TABLE VII: Token-mode A/B study (full vs. balanced).

Task	full tok.	bal. tok.	Δ	SCORE (f→b)
vecadd	68,279	46,924	−31.3%	1.000 → 1.000
matmul	55,357	50,729	−8.4%	2.270 → 2.691
dotProduct	67,253	48,146	−28.4%	3.000 → 3.000

Balanced mode saved 8–31% tokens (mean 22.7%) with no score degradation. The score–token frontier: vecadd full 68k → 1.000; balanced 47k → 1.000; aggressive 17k (−75%) → 0.737. Strategy-proposal reasoning is where optimization quality comes from.

TABLE VIII: Archived repeat runs (DeepSeek V4 Pro).

Task (runs)	SCOREs	Latencies	Tokens
projection (3)	1.400 / 1.400 / 1.400	0 / 0 / 0	3.6k / 3.8k / n.a.
residual (3)	3.433 / 3.925 / 4.000	32 / 18 / 10	34.5k / 57.4k / 7.8k
dotProduct (2)	3.000 / 3.000	20 / 36	67.3k / 48.1k
matmul (2)	2.270 / 2.691	10,881 / 3,126	55.4k / 50.7k
fir (2)	2.000 / 2.000	1,054 / 542	n.a. / 127.2k
vecadd (3)	1.000 / 1.000 / 0.737	20 / 38 / 4,117	68.3k / 46.9k / 17.4k

Three findings: (1) correctness is stable (15/15 runs passed all gates); (2) optimization quality varies with strategy randomness (residual latency spans 10–32 cycles); (3) the one sub-par score (vecadd 0.737) came from aggressive token-mode, not model randomness.

H. dotProduct (Optimize, Perfect Score)

Latency reduced from 73 to 20 cycles over 4 rounds. The reviewer AI rejected “Full Unroll to 1024 Lanes” (resource explosion) and “Increase to 128” (pragma conflict).

I. residual (Structural, Cosim Deadlock Repair)

Hardest task (difficulty 4). DATAFLOW skip connection: producer writes entire main stream before skip stream; co-simulation’s depth-2 FIFOs cannot buffer the burst, causing deadlock. The pre-csim review fixed this before spending

```

1 #include "dotProduct.h"
2 FeatureType dotProduct(
3     FeatureType param[NUM_FEATURES],
4     DataType feature[NUM_FEATURES]) {
5     const int unroll_factor = PAR_FACTOR;
6     #pragma HLS array_partition variable=param \
7         cyclic factor=unroll_factor
8     #pragma HLS array_partition variable=feature \
9         cyclic factor=unroll_factor
10    FeatureType result = 0;
11    DOT:
12    for (int i = 0; i < NUM_FEATURES/PAR_FACTOR; i++)
13    {
14        #pragma HLS PIPELINE II=1
15        DOT_INNER:
16        for (int j = 0; j < PAR_FACTOR; j++)
17            result += param[i*PAR_FACTOR+j]
18                * feature[i*PAR_FACTOR+j];
19    }
20    return result;

```

Listing 1: Optimized dotProduct kernel (51.35× acceleration).

cosim credits. Latency dropped from 68 to 10 cycles over 4 optimization rounds; post-optimization cosim passed, achieving 4.000.

J. projection (Repair, Minimal Flow)

Bug: dropped vertex term in angle==0 branch. Pre-csim review caught it before the first csim credit. Entire run: 5 credits, 3,629 tokens. Qwen3.6’s reviewer rejected its own repairs twice for a missing default case (angle==3) before accepting a hardened version.

SWE-agent [10] shows agent–computer interface matters; our feedback distiller shapes how the LLM perceives Vitis output. Anthropic’s guidance [11] recommends simple workflows—our linear-with-backtracking flow is deliberately a workflow with LLM steps, not a free-running planner. AutoChip [2] generates HDL with compiler feedback; RTL-Fixer [12] repairs RTL syntax with retrieval-augmented debugging. Our KB pattern entries are distilled from HLS-Repair, AutoChip, and RTL-Fixer [3], [2], [12].

K. Design Decisions and Rationale

L. Latency-First PPA Is Score-Optimal

The scored PPA term is $p = \min(\text{Accel}, 8)/8$ —latency-only. Under this metric, latency-first optimization is score-optimal. We guard against area explosion (reviewer-AI rejects

Decision	Rationale
Fork harness, don't rewrite	Evaluation interface is locked to in-process calls
Dual-gate review	LLM self-check misses signature changes; mechanical gate backs it
Monotone checkpoint	Score layering maps naturally; failed candidates auto-rollback
Design brief before optimize	Without design context, LLM gives only generic advice
Dual-confirmation for combos	Pragma interaction is an LLM high-error point
Re-verify csim before synth	1 credit < 4 credits; fixing synth can break correctness
Real rollback on cosim fail	Previously only logged; would ship with deadlock

resource-explosion proposals) and timing failure (5 ns synthesis target; Fmax 200–353 MHz observed).

M. A Harness Edge Case: Zero Latency

The projection kernel synthesizes to combinational logic (latency 0). The official scorer computes acceleration only if `cand_lat` and `base_lat:—0` is falsy—so a latency-0 design forfeits the 0.3 PPA slice, capping at $2 \times 0.7 = 1.400$.

N. Limitations

- 1) Single-run protocol per (model, task) for Qwen models; cross-model conclusions are indicative, not statistical.
- 2) Pre-csim review can corrupt a good baseline (observed once: Qwen3.5 on `matmul`, latency doubled). Anchor archive on original baseline first (future work).
- 3) No backoff on synth timeout (Qwen3.5 on `vecadd`, $4\times$).
- 4) No HTTP-level retry; one Qwen3.5 run lost to read timeout.
- 5) Aggressive mode trade-off: `vecadd` 1.000→0.737.
- 6) KB not naturally triggered on six tasks (correctness passed first try).
- 7) Latency variance: residual 10–32 cycles across runs.
- 8) No generate-type task in the suite.

O. Future Work

Anchor archive on unmodified baseline; synth-timeout backoff; HTTP-level retry; functional-pattern retrieval; KB ablation; area/power-aware optimization; hidden test set validation.

The knowledge base is a 22-entry bug→fix corpus in `entries.json`. Retrieval: case-insensitive substring matching on short error-code/keyword signatures.

Full verbatim prompts archived as `prompts.jsonl` per run in `experiments/`.

a) System prompt (all repair/review calls):

You are a senior Vitis 2025.2 HLS engineer targeting Alveo U55C @ 200 MHz. Keep designs functionally correct first, optimized second. Prefer general HLS techniques (PIPELINE, UNROLL, ARRAY_PARTITION, DATAFLOW) ...

b) Reviewer AI rejection (Qwen3.6 27B, projection):

FAIL

```
- **Missing default case for 'angle == 3': 'bit2' implies
  a 0-3 range. The code lacks an 'else'/default branch,
  leaving 'triangle_2d' uninitialized when 'angle == 3'.
  This causes undefined behavior ...
```

TABLE IX: Knowledge-base corpus breakdown (22 entries).

Category	#	Coverage
synth	5	dataflow conflict, II scheduling fail, array-port conflict, pragma same-level conflict, pragma file-scope
csim	4	test-case failed, compile error, numeric reorder tolerance, boundary condition
cosim	6	FIFO-burst deadlock, FIFO depth, TLAST missing, TVALID/TREADY handshake, ap_ctrl hang, synth/cosim latency mismatch
pattern	6	retrieve-before-repair, correctness/optimization separation, syntax-first, error-message pairing, expected-value-diff feedback, bounded feedback loop
other	1	DATA_PACK interface misuse

TABLE X: Mapping from the 10 submission rules [4] to evidence.

#	Rule	Evidence
1	Alveo U55C target for cosim	Table V (part xcu55c)
2	Vitis 2025.2	csynth reports in archive
3	Pass csim, cosim, synth + report	Tables I, II
4	Hardware runs at ≥ 100 MHz	Table V: Fmax 200–353 MHz
5	Token consumption as eval factor	§G; token columns in tables
6	Evaluate three recommended models	§III-B, Table II
7	Hidden test benchmarks	All grading via official hidden testbenches
8	Docker build/run + Dockerfile	Appendix O0b
9	Source, testbenches, repro materials	Submission archive; experiments/ data
10	≤ 5 -min demo video	Separate deliverable (TUI dashboard)

c) *Docker*.: We provide a Dockerfile and `docker-run.sh`. Image: Ubuntu 22.04 + Vitis dependency layer + Python 3.12 + agent source + harness. Vitis is not baked in (~92 GB); it is read-only mounted from the host. Key finding: the *entire* Xilinx tree must be mounted to the *same path* in the container (`settings64.sh` hard-codes sibling paths).

d) Commands.:

```
# Setup (interactive)
python3 scripts/setup_env.py
python3 scripts/check_env.py --test-api

# Build + run (Docker)
docker build -t fpga-agent:latest .
./docker-run.sh \
  contest/fpt26-harness/tasks/projection_bugfix \
  --backend deepseek

# Model selection via env overrides
LLM_BASE_URL=<endpoint> LLM_MODEL=qwen3.6-27b \
LLM_THINKING=enable_thinking \
python3 scripts/run_agent.py <task_dir> --backend deepseek
```


f) *TUI dashboard.*: Textual/Rich terminal dashboard: five-stage flow chart, tool-error bar (doubles as strategy panel during optimization), streaming LLM output (CoT in dim italic, C++ syntax-highlighted), status bar with credit/token progress.

